Programowanie Funkcyjne

Marcel Kuźma i Joanna Żupnik

Wprowadzenie do programowania funkcyjnego

Czym jest programowanie funkcyjne (FP)?

Programowanie funkcyjne rozkłada problem na zestaw funkcji. Funkcje przyjmują dane wejściowe i zwracają wynik, a ich działanie powinno zależeć tylko od przekazanych argumentów, bez zmiany stanu programu i innych danych w programie.

Różne paradygmaty programowania

Funkcyjny (FP)

```python
def pole_prostokata(a,b):
    return a * b
print(pole_prostokata(5,5))
```

Obiektowy (OOP)

```python
class Prostokat:
    def __init__(self,a,b):
        self.a = a
        self.b = b

    def pole(self):
        return self.a * self.b

prostokat = Prostokat(5,5)
print(prostokat.pole())
```

Proceduralny

```python
a = 5
b = 5
pole = a * b
print(pole)
```

	imperatywne	funkcyjne
Główna idea	Opisuje **jak** wykonać zadanie krok po kroku	Opisuje **co** ma powstać, **co** chcemy osiągnąć
Podejście do stanu	Stan programu ulega ciągłym zmianom (zmienne, listy i słowniki są bezpośrednio nadpisywane)	Stan jest niezmienny – dane wejściowe nie są modyfikowane, wyniki są nowymi obiektami.
Przepływ sterowania	Pętle (for, while), instrukcje warunkowe (if/else).	Rekurencja, wywołania funkcji (np. map, filter, reduce).
Rodzaj funkcji	Mają efekty uboczne(modyfikacja zmiennych, operacje I/O)	Czyste funkcje (pure functions)
Testowanie	Czasem trudniejsze przez stan i efekty uboczne	zwykle łatwiejsze, ponieważ czysta funkcja zawsze da ten sam wynik dla tych samych danych.
Kod końcowy	Często dłuższy i podatny na efekty uboczne, nieoczekiwane zmiany stanu	krótszy, czytelniejszy oraz łatwiejszy do optymalizacji kosztem nadmiernego tworzenia obiektów

Imperatywne

```python
transakcje = [1500, -50, 300, -120, 4000]

def pobierz_wplywy(lista_transakcji):  1 us
    wynik = []
    for kwota in lista_transakcji:
        if kwota > 0:
            wynik.append(kwota)
    return wynik

wplywy = pobierz_wplywy(transakcje)
```

Wykorzystanie pętli i warunku. **JAK** ma wykonać zadanie: zrób pętlę, weź element, sprawdź warunek, dodaj do listy.

Funkcyjne

```python
transakcje = [1500, -50, 300, -120, 4000]

wplywy = list(filter(lambda kwota: kwota > 0, transakcje))
```

Skupia się na deklarowaniu samego celu, czyli **CO** ma wykonać

Imperatywne

```
transakcje = [1500, -50, 300, -120, 4000]

def pobierz_wplywy(lista_transakcji):  1 us
    wynik = []
    for kwota in lista_transakcji:
        if kwota > 0:
            wynik.append(kwota)
    return wynik

wplywy = pobierz_wplywy(transakcje)
```

Tworzymy pusty stan i ciągle go mutujemy

Funkcyjne

```
transakcje = [1500, -50, 300, -120, 4000]

wplywy = list(filter(lambda kwota: kwota > 0, transakcje))
```

Od razu otrzymujemy nową zmienną

Python w programowaniu funkcyjnym

Python nie jest językiem czysto funkcyjnym. Wspiera głównie programowanie imperatywne i obiektowe. Jednak posiada on potężny zestaw narzędzi, który pozwala programistom pisać kod w stylu funkcyjnym. Ponieważ język sam nie wymusza na nas braku skutków ubocznych, styl funkcyjny w Pythonie wymaga dyscypliny i dobrego zaplanowania.

Elementy, które pozwalają na programowanie funkcyjne w Pythonie:
-Funkcje czyste
-Funkcje pierwszej klasy
-Lambda
-Funkcja wyższego rzędu
-Rekurencja

Funkcja czysta

Aby program był łatwy do zrozumienia, należy pozbawić go skutków ubocznych. Służą do tego funkcje czyste, które opierają się na dwóch zasadach:

1. Są deterministyczne: Dla takich samych argumentów wejściowych zawsze zwrócą ten sam wynik.
2. Brak skutków ubocznych: Funkcja nie modyfikuje argumentów wejściowych ani nie oddziałuje na zewnętrzny stan programu. Jedynym śladem ich działania jest wartość, którą zwracają (tzw. return).

Przykład funkcji “nieczystej”

```
zdrowie = 100

def otrzymaj_cios_nieczysta(cios):  2 usages
    global zdrowie
    zdrowie = zdrowie - cios
    return zdrowie

print(otrzymaj_cios_nieczysta(20)) #80
print(otrzymaj_cios_nieczysta(20)) #60
```

Funkcja modyfikuje zmienną globalną

Podaliśmy dwa razy ten sam argument, a funkcja zwórciła dwa różne wyniki. Brak determinizmu

Przykład funkcji czystej

```
zdrowie = 100

def otrzymaj_cios_czysta(zdrowie_gl,cios):  2 usages
    nowe_zdrowie = zdrowie_gl - cios
    return nowe_zdrowie

print(otrzymaj_cios_czysta(zdrowie, cios: 20)) #80
print(otrzymaj_cios_czysta(zdrowie, cios: 20)) #80
```

Funkcja nie modyfikuje zmiennej globalnej.Brak skutów ubocznych

Otrzymujemy zawsze ten sam wynik dla danego argumentu, czyli jest deterministyczna

Funkcje pierwszej klasy

W językach takich jak Python funkcje są traktowane dokładnie tak samo, jak wszystkie inne dane (np. liczby czy tekst). Mówimy wtedy, że są obiektami pierwszej klasy. Dzięki temu język pozwala na bardzo elastyczne zarządzanie nimi.

Właściwości takiej funkcji :

-Funkcja to obiekt: Każda funkcja jest obiektem przechowywanym w pamięci komputera.

-Można przypisać funkcję do zwykłej zmiennej.

-Można wrzucać funkcje do list, słowników czy innych struktur danych.

```python
def powitanie(imie):  1 usage
    return f"Cześć, {imie}"

zmienna_powitanie = powitanie
print(zmienna_powitanie("Darek"))
```

```python
def dodaj(a, b): return a + b  1 usage
def odejmij(a, b): return a - b  1 usage
kalkulator = {
"plus": dodaj,
"minus": odejmij
}
print(kalkulator["plus"](a: 10, b: 5))
print(kalkulator["minus"](a: 10, b: 5))
```

Funkcja anonimowa (lambda)

Czym jest funkcja anonimowa?

Funkcja anonimowa, nazywana również wyrażeniem lambda, to niewielka funkcja, która nie posiada nazwy. Składa się z jednej linijki kodu. Może przyjmować dowolną liczbę argumentów, a jej działanie jest określone w jednym wyrażeniu.

Składnia:

```
lambda argumenty: wyrażenie
```

Przykłady:

```
podnies_do_kwadratu = lambda x: x ** 2
print(podnies_do_kwadratu(4))
#16
```

Można przypisać do zmiennej

```
dodaj = lambda x, y: x + y
print(dodaj(3, 5))
#8
```

Może przyjmować dowolną liczbe argumentów

```
liczby = [1, 2, 3, 4, 5]
powiekszone_liczby = list(map(lambda x: x * 2, liczby))
print(powiekszone_liczby)
#[2, 4, 6, 8, 10]
```

Można użyć z innymi funkcjami

Funkcje wyższego rzędu

Czym jest funkcja wyższego rzędu?

Funkcja wyższego rzędu to funkcja, która przyjmuje inne funkcje jako argumenty lub zwraca funkcję jako wynik.

Przyjmuje funkcję jako agrument

```python
def wykonaj_operacje(liczby,operacja):
    wynik = []
    for liczba in liczby:
        wynik.append(operacja(liczba))
    return wynik

def powieksz_razy_2(x):
    return x * 2

def podnies_do_kwadratu(x):
    return x ** 2

liczby = [1, 2, 3, 4, 5]

print(wykonaj_operacje(liczby, powieksz_razy_2))
#[2, 4, 6, 8, 10]
print(wykonaj_operacje(liczby, podnies_do_kwadratu))
#[1, 4, 9, 16, 25]
```

Zwraca funkcję jako wynik

```python
def stworz_dodawanie(n):
    def dodaj(x):
        return x + n
    return dodaj


dodaj5 = stworz_dodawanie(5)
dodaj10 = stworz_dodawanie(10)

print(dodaj5(3))
#8
print(dodaj10(3))
#13
```

Funkcje wyższego rzędu

map(funkcja, element_iterowalny)

Funkcja wbudowana w Pythonie , która służy do zastosowania podanej funkcji dla każdego elementu iterowalnego.

```
liczby  = [1, 2, 3, 4, 5]

def powieksz_razy_2(x):
    return x * 2

powiekszone_liczby = map(powieksz_razy_2, liczby)

print(powiekszone_liczby)
#<map object at 0x00000240F6A3B820>

print(list(powiekszone_liczby))
#[2, 4, 6, 8, 10]

print(list(powiekszone_liczby))
#[] (pusta lista ponieważ map został już wyczerpany)
```

lazy evaluation

wynik

jednorazowość

Skrócony zapis z użyciem lambdy:

```
liczby = [1, 2, 3, 4, 5]
powiekszone_liczby = list(map(lambda x: x * 2, liczby))
print(powiekszone_liczby)
#[2, 4, 6, 8, 10]
```

Funkcje wyższego rzędu

filter(funkcja, element iterowany)

Funkcja wbudowana w Pythonie, która służy do filtrowania elementów z obiektu iterowalnego. Przepuszcza tylko te elementy, dla których podana funkcja zwraca wartość True.

```
liczby  = [1, 2, 3, 4, 5]

def czy_parzysta(x):
    return x % 2 == 0

parzyste_liczby = filter(czy_parzysta, liczby)

print(parzyste_liczby)
#<filter object at 0x0000014178D0B8E0>

print(list(parzyste_liczby))
#[2, 4]

print( list(parzyste_liczby))
#[] (pusta lista ponieważ filter został już wyczerpany)
```

Skrócony zapis z użyciem lambdy:

```
liczby = [1, 2, 3, 4, 5]
parzyste_liczby = list(filter(lambda x: x % 2 == 0, liczby))
print(parzyste_liczby)
#[2, 4]
```

Funkcje wyższego rzędu

reduce(funkcja, element iterowany)
Funkcja w Pythonie (z modułu functools), która służy do redukcji elementów iterowalnych do jednej wartości. Łączy kolejne elementy, stosując podaną funkcję.

```
from functools import reduce
liczby = [1, 2, 3, 4]

def dodawanie(x, y):
    return x + y

suma = reduce(dodawanie, liczby)
print(suma)
#10
```

Skrócony zapis z użyciem lambdy:

```
from functools import reduce
liczby = [1, 2, 3, 4]
suma_lambda = reduce(lambda x, y: x + y, liczby)
print(suma_lambda)
#10
```

Funkcje wyższego rzędu

sorted(element iterowany,funkcja)

Funkcja wbudowana w Pythonie, która służy do sortowania elementów iterowalnych. Można użyć argumentu key, aby określić funkcję, według której ma nastąpić sortowanie.

```
jezyki = ["python", "javascript", "java", "c++"]

def sortowanie_po_dlugosci(x):
    return len(x)

slowa_posortowane = sorted(jezyki, key=sortowanie_po_dlugosci)
print(slowa_posortowane)
# ['c++', 'java', 'python', 'javascript']
```

Skrócony zapis z użyciem lambdy:

```
jezyki = ["python", "javascript", "java", "c++"]
slowa_posortowane = sorted(jezyki, key=lambda x: len(x))
print(slowa_posortowane)
# ['c++', 'java', 'python', 'javascript']
```

Rekurencja

Czym jest rekurencja?

Mechanizm programistyczny polegający na wywoływaniu funkcji przez samą siebie w celu rozwiązania mniejszego fragmentu tego samego problemu. Działa poprzez dzielenie złożonego zadania na prostsze etapy, aż do osiągnięcia przypadku bazowego, który zatrzymuje dalsze wywołania.

```python
def silnia(n):
    if n == 0: # Przypadek bazowy: silnia z 0 to 1
        return 1

    return n * silnia(n - 1)  # krok rekurencyjny

print(silnia(4))  # 24
```

1. Schodzienie w dół

- Funkcja wywołuje samą siebie wielokrotnie
- Każde wywołanie czeka na wynik wywołania podrzędnego
- Działanie do momentu osiągnięcia warunku zakończenia

silnia(4) → 4 * silnia(3)
silnia(3) → 3 * silnia(2)
silnia(2) → 2 * silnia(1)
silnia(1) → 1 * silnia(0)
silnia(0) → warunek zakończenia → zwraca 1

2. Wspinanie w góre

- Funkcja zaczyna zwracać wyniki w odwrotnej kolejności
- Każde wywołanie używa wyniku wywołania podrzędnego, aby obliczyć swoją wartość

silnia(1) → 1 * 1 = 1
silnia(2) → 2 * 1 = 2
silnia(3) → 3 * 2 = 6
silnia(4) → 4 * 6 = 24

Główne obszary zastosowań

- **Współbieżność i komunikatory (np. WhatsApp, Discord)**

Dlaczego warto: Bezpieczna obsługa milionów jednoczesnych połączeń bez konfliktów danych i zawieszania serwerów.
Popularne języki: Erlang, Elixir.

- **Big Data i analityka danych**

Dlaczego warto: Łatwe i bezpieczne zrównoleglanie obliczeń na setkach serwerów w celu przetwarzania terabajtów informacji.
Popularne języki: Scala, Clojure.

- **FinTech, giełda i systemy krytyczne**

Dlaczego warto: Matematyczna precyzja i w 100% przewidywalny kod. Brak "skutków ubocznych" eliminuje kosztowne błędy finansowe.
Popularne języki: Haskell, OCaml, F#.

- **Nowoczesny Frontend (aplikacje webowe)**

Dlaczego warto: Niezmienność stanu pozwala utrzymać porządek i przewidywalność w bardzo skomplikowanych, dynamicznych interfejsach użytkownika.
Popularne języki: JavaScript / TypeScript (np. z biblioteką React), ClojureScript.

Dlaczego stosujemy programowanie funkcyjne?

- **Czytelność i zwięzłość kodu**: Podejście funkcyjne pozwala pisać krótszy kod, który skupia się na deklarowaniu celu (co ma powstać), a nie na opisywaniu instrukcji krok po kroku.
- **Bezbolesne testowanie:** Dzięki „czystym funkcjom”, które zawsze dają ten sam wynik dla tych samych danych, testowanie staje się znacznie łatwiejsze i bardziej przewidywalne.
- **Bezpieczeństwo danych:** Funkcje nie modyfikują zmiennych globalnych ani argumentów wejściowych, co eliminuje błędy wynikające z nieoczekiwanych zmian stanu programu.
- **Efektywne przetwarzanie zbiorów:** Za pomocą narzędzi takich jak map, filter i reduce można błyskawicznie transformować i filtrować duże ilości danych bez użycia skomplikowanych pętli.
- **Łatwiejsza optymalizacja:** Kod funkcyjny, mimo że czasem tworzy więcej obiektów, jest łatwiejszy do optymalizacji pod kątem wydajności niż długi kod imperatywny.
- **Rozwiązywanie złożonych problemów:** Mechanizm rekurencji pozwala dzielić trudne zadania na mniejsze, prostsze etapy, aż do osiągnięcia celu.

Bibliografia

- Steven F. Lott, “Functional Python Programming, Second Edition”
- A. M. Kuchling “Functional Programming HOWTO” Python 3 Documentation. Dostępne na https://docs.python.org/3/howto/functional.html
- GeeksforGeeks “Functional Programming in Python” . Dostępne na: https://www.geeksforgeeks.org/python/functional-programming-in-python/
- Sturtz, J. “Functional Programming in Python: When and How to Use It. Real Python.”Dostępne na: https://realpython.com/python-functional-programming/
- Łukasz Trzcianowski, Patryk Szypulski “Praktycznie o programowaniu funkcyjnym w Pythonie”. Dostępne na “https://www.youtube.com/watch?v=q_TrxiAC72w”